



Preparación del Sistema POS - CRTLPyme

Fecha: 25 de Octubre, 2025
Versión: 1.0.0 (MVP)
Estado: Análisis Completo para Desarrollo POS



Ubicación del Proyecto

Ruta Principal

```
/home/ubuntu/github_repos/crtlpyme-mvp-temp/
```

Proyecto Completo

- **Nombre:** CRTLPyme (pos-saas-chile)
- **Framework:** Next.js 15.0.3
- **React:** 19.0.0
- **ORM:** Prisma 6.0.1
- **Base de Datos:** PostgreSQL (Supabase)
- **UI:** Radix UI + Tailwind CSS + shadcn/ui



Estructura del Proyecto

Estructura de Carpetas Principal

crtlpyme-mvp-temp/	
app/	# App Router de Next.js 15
admin/	# Rutas administrativas protegidas
dashboard/	# Dashboard principal
page.tsx	# ☒ Implementado
inventory/	# Gestión de inventario
page.tsx	# ☒ Implementado (CRUD completo)
api/	# API Routes
auth/	# Endpoints de autenticación
[...nextauth]/	# NextAuth.js configurado
register/	# Registro de usuarios
init-db/	# Inicialización de DB
products/	# CRUD de productos
route.ts	# GET (list), POST (create)
[id]/	# GET , PUT , DELETE por ID
auth/	# Páginas de autenticación
login/	# ☒ Login implementado
register/	# Registro de usuarios
layout.tsx	# Layout principal
page.tsx	# Página de inicio
components/	# Componentes reutilizables
charts/	# Componentes de gráficos
sales-chart.tsx	# Gráfico de ventas
dashboard/	# Componentes del dashboard
metric-card.tsx	# Tarjetas de métricas
layout/	# Componentes de layout
dashboard-layout.tsx	# Layout del dashboard
ui/	# 54 componentes shadcn/ui
button.tsx	
card.tsx	
dialog.tsx	
form.tsx	
input.tsx	
select.tsx	
table.tsx	
toast.tsx	
... (51 más)	
providers.tsx	# Provider de sesión
theme-provider.tsx	# Provider de tema
lib/	# Utilidades y configuración
auth.ts	# Configuración de NextAuth
prisma.ts	# Cliente de Prisma
utils.ts	# Utilidades generales
prisma/	# Configuración de Prisma
schema.prisma	# ☒ Schema completo definido
seed.ts	# Script de seed con datos demo
hooks/	# Custom React Hooks
data/	# Datos de ejemplo
productos_chilenos.json	# 100+ productos chilenos
docs/	# Documentación del proyecto
Fase 1/	# Documentación Fase 1
Fase 2/	# Documentación Fase 2
Fase 3/	# Documentación Fase 3
.env	# Variables de entorno
package.json	# Dependencias del proyecto
middleware.ts	# ☒ Protección de rutas
components.json	# Configuración shadcn/ui

Rutas Implementadas

✓ Rutas Funcionales

Ruta	Descripción	Estado	Protegida
/	Página de inicio	✓ Implementado	No
/auth/login	Login de usuarios	✓ Implementado	No
/auth/register	Registro de usuarios	✓ Implementado	No
/admin/dashboard	Dashboard principal	✓ Implementado	✓ Sí
/admin/inventory	Gestión de inventario	✓ Implementado	✓ Sí
/api/auth/[...nextAuth]	Autenticación NextAuth	✓ Implementado	No
/api/products	CRUD de productos	✓ Implementado	✓ Sí

✗ Rutas Pendientes (POS)

Ruta Sugerida	Descripción	Prioridad
/admin/pos	Punto de venta principal	● Alta
/admin/pos/checkout	Proceso de pago	● Alta
/admin/sales	Historial de ventas	● Media
/admin/sales/[id]	Detalle de venta	● Media
/admin/cash-session	Gestión de caja	● Alta
/api/sales	CRUD de ventas	● Alta
/api/cash-session	Gestión de sesiones de caja	● Alta

Componentes UI Disponibles

El proyecto tiene **54 componentes** de shadcn/ui listos para usar:

Componentes Esenciales para POS

Componente	Ubicación	Uso en POS
Button	components/ui/button.tsx	Botones de acciones
Card	components/ui/card.tsx	Tarjetas de productos
Dialog	components/ui/dialog.tsx	Modales de confirmación
Form	components/ui/form.tsx	Formularios de pago
Input	components/ui/input.tsx	Búsqueda de productos
Select	components/ui/select.tsx	Selección de métodos de pago
Table	components/ui/table.tsx	Lista de productos en carrito
Badge	components/ui/badge.tsx	Estados (stock, descuentos)
Toast	components/ui/toast.tsx	Notificaciones de éxito/error
Skeleton	components/ui/skeleton.tsx	Estados de carga
Separator	components/ui/separator.tsx	Separadores visuales
ScrollArea	components/ui/scroll-area.tsx	Áreas con scroll
Command	components/ui/command.tsx	Búsqueda rápida de productos
Popover	components/ui/popover.tsx	Menús contextuales
Calendar	components/ui/calendar.tsx	Selección de fechas
Checkbox	components/ui/checkbox.tsx	Opciones múltiples
Alert	components/ui/alert.tsx	Alertas de stock bajo

Componentes Personalizados Disponibles

Componente	Ubicación	Descripción
MetricCard	components/dashboard/ metric-card.tsx	Tarjetas de métricas para es- tadísticas
SalesChart	components/charts/sales- chart.tsx	Gráfico de ventas con Re- charts
DashboardLayout	components/layout/dash- board-layout.tsx	Layout con navegación

Esquema de Base de Datos Actual

Configuración de Conexión

```
# Variables de entorno (.env)
DATABASE_URL="postgresql://postgres.bxfetsflhxhigacutfe:Pyme_2025@aws-0-us-  
east-1.pooler.supabase.com:6543/postgres"
```

Nota: El sistema usa **Prisma ORM** sobre **Supabase PostgreSQL**.



Tablas Existentes (Schema Prisma)



Tablas Ya Implementadas

1. Tenant (Multi-tenencia)

```

model Tenant {
  id          String   @id @default(cuid())
  businessName String
  rut          String   @unique
  email        String   @unique
  phone        String?
  address      String?
  isActive     Boolean  @default(true)
  planType     PlanType @default(BASIC)
  maxCashiers  Int       @default(2)
  extraCashiers Int      @default(0)
  createdAt    DateTime  @default(now())
  updatedAt    DateTime  @updatedAt

  // Relaciones
  users          User[]
  products       Product[]
  sales          Sale[]
  cashSessions   CashSession[]
  fixedExpenses  FixedExpense[]
  stockAdjustments StockAdjustment[]
  auditLogs      AuditLog[]
}

```

Estado:  Tabla necesaria para POS (multi-tenencia)

2. User (Usuarios)

```

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  password    String
  firstName   String
  lastName    String
  role        UserRole
  isActive    Boolean   @default(true)
  tenantId    String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  // Relaciones
  tenant       Tenant    @relation(fields: [tenantId], references: [id])
  sales        Sale[]
  cashSessions CashSession[]
  stockAdjustments StockAdjustment[]
  auditLogs    AuditLog[]
}

enum UserRole {
  PROVEEDOR // Administrador SaaS
  ADMIN     // Administrador Cliente
  CAJA      // Operador punto de venta ★ ESENCIAL PARA POS
  INVENTARIO // Encargado de stock
  SOPORTE   // Soporte técnico
}

```

Estado: ☒ Tabla necesaria para POS (usuario de caja)

3. Product (Productos/Inventario)

```

model Product {
  id          String    @id @default(cuid())
  sku         String
  barcode     String?
  name        String
  description  String?
  category    String
  brand       String?
  costPrice   Decimal    @db.Decimal(10,2)
  salePrice   Decimal    @db.Decimal(10,2)
  stock       Int        @default(0)
  minStock    Int        @default(5)
  isActive    Boolean    @default(true)
  tenantId    String
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  // Relaciones
  tenant       Tenant    @relation(fields: [tenantId], references: [id])
  saleItems    SaleItem[]
  stockAdjustments StockAdjustment[]
}

```

Estado:  Implementado con CRUD completo

API: /api/products funcional

Interfaz: /admin/inventory implementada

4. Sale (Ventas) ★ TABLA CRÍTICA PARA POS

```
model Sale {
  id          String      @id @default(cuid())
  saleNumber  String      // número consecutivo por tenant
  subtotal    Decimal     @db.Decimal(10,2)
  tax         Decimal     @db.Decimal(10,2) @default(0)
  total       Decimal     @db.Decimal(10,2)
  paymentMethod PaymentMethod
  cashReceived Decimal?   @db.Decimal(10,2)
  change      Decimal?   @db.Decimal(10,2)
  status      SaleStatus @default(COMPLETED)
  userId      String
  tenantId    String
  cashSessionId String?
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  // Relaciones
  user      User      @relation(fields: [userId], references: [id])
  tenant    Tenant    @relation(fields: [tenantId], references: [id])
  cashSession CashSession? @relation(fields: [cashSessionId], references: [id])
  items     SaleItem[]
}

enum PaymentMethod {
  CASH      // Efectivo
  DEBIT     // Débito
  CREDIT    // Crédito
  TRANSFER  // Transferencia
}

enum SaleStatus {
  PENDING
  COMPLETED
  CANCELLED
}
```

Estado:  Schema definido,  API no implementada

Prioridad:  Alta - Necesaria para POS

5. SaleItem (Items de Venta) ★ TABLA CRÍTICA PARA POS

```
model SaleItem {
  id          String   @id @default(cuid())
  quantity    Int
  unitPrice   Decimal  @db.Decimal(10,2)
  unitCost    Decimal  @db.Decimal(10,2)
  subtotal    Decimal  @db.Decimal(10,2)
  saleId      String
  productId   String
  tenantId    String

  // Relaciones
  sale      Sale    @relation(fields: [saleId], references: [id])
  product   Product @relation(fields: [productId], references: [id])
}
```

Estado: ✅ Schema definido, ❌ API no implementada

Prioridad: 🔴 Alta - Necesaria para POS

6. CashSession (Sesión de Caja) ★ TABLA CRÍTICA PARA POS

```
model CashSession {
  id          String           @id @default(cuid())
  initialAmount Decimal        @db.Decimal(10,2)
  finalAmount  Decimal?        @db.Decimal(10,2)
  expectedAmount Decimal?      @db.Decimal(10,2)
  difference    Decimal?        @db.Decimal(10,2)
  status       CashSessionStatus @default(OPEN)
  openedAt     DateTime         @default(now())
  closedAt     DateTime?
  userId       String
  tenantId     String

  // Relaciones
  user      User    @relation(fields: [userId], references: [id])
  tenant    Tenant  @relation(fields: [tenantId], references: [id])
  sales     Sale[]

  enum CashSessionStatus {
    OPEN
    CLOSED
  }
}
```

Estado: ✅ Schema definido, ❌ API no implementada

Prioridad: 🔴 Alta - Necesaria para POS

7. StockAdjustment (Ajustes de Inventario)

```

model StockAdjustment {
  id          String          @id @default(cuid())
  productId   String
  quantity    Int             // puede ser negativo para mermas
  type        AdjustmentType
  reason      String?
  userId      String
  tenantId    String
  createdAt   DateTime        @default(now())

  // Relaciones
  product Product @relation(fields: [productId], references: [id])
  user    User    @relation(fields: [userId], references: [id])
  tenant  Tenant  @relation(fields: [tenantId], references: [id])
}

enum AdjustmentType {
  PURCHASE // Compra
  LOSS     // Merma
  CORRECTION // Corrección de inventario
  RETURN   // Devolución
}

```

Estado: ☒ Schema definido, ☐ No implementado

Prioridad: ● Media - Útil para auditoría de inventario

8. FixedExpense (Gastos Fijos)

```

model FixedExpense {
  id          String          @id @default(cuid())
  name        String
  amount      Decimal         @db.Decimal(10,2)
  frequency    ExpenseFrequency
  isActive     Boolean         @default(true)
  tenantId     String
  createdAt    DateTime        @default(now())
  updatedAt    DateTime        @updatedAt

  // Relaciones
  tenant Tenant @relation(fields: [tenantId], references: [id])
}

enum ExpenseFrequency {
  DAILY
  WEEKLY
  MONTHLY
  YEARLY
}

```

Estado: ☒ Schema definido, ☐ No implementado

Prioridad: ● Baja - Para reportes de rentabilidad

9. AuditLog (Auditoría)

```
model AuditLog {
  id          String    @id @default(cuid())
  action      String    // CREATE, UPDATE, DELETE
  entity      String    // tabla afectada
  entityId    String    // ID del registro
  oldValues   Json?
  newValues   Json?
  userId      String?
  tenantId    String
  createdAt   DateTime @default(now())

  // Relaciones
  user        User? @relation(fields: [userId], references: [id])
  tenant      Tenant @relation(fields: [tenantId], references: [id])
}
```

Estado: ☒ Schema definido, ☒ No implementado

Prioridad: ☒ Baja - Para auditoría avanzada



Datos de Ejemplo Disponibles

Script de Seed

Ubicación: prisma/seed.ts

Contenido:

- ☒ 1 Tenant demo: "Demo Chile SpA"
- ☒ 1 Usuario admin: admin@demo.cl / Demo123!
- ☒ 100+ productos chilenos desde data/productos_chilenos.json

Ejecutar seed:

```
cd /home/ubuntu/github_repos/crtlpyme-mvp-temp
npm run seed
```

Datos de Productos

Archivo: data/productos_chilenos.json

Categorías disponibles:

- Bebidas (gaseosas, jugos)
- Alimentos (snacks, conservas)
- Lácteos
- Panadería
- Dulces
- Limpieza
- Y más...

Estructura de cada producto:

```
{
  "name": "Coca-Cola Original 1.5L",
  "brand": "Coca-Cola",
  "category": "bebidas",
  "subcategory": "gaseosas",
  "ean13": "7804123456789",
  "price_clp": 1520
}
```

 Tablas Necesarias para el Sistema POS

 Tablas Ya Existentes

Tabla	Estado Schema	Estado API	Estado UI	Uso en POS
Tenant	 Definido	 Parcial	 No	Multi-tenencia
User	 Definido	 Parcial	 Login	Operador de caja
Product	 Definido	 Completo	 Completo	Búsqueda de productos
Sale	 Definido	 No	 No	★ Crítico
SaleItem	 Definido	 No	 No	★ Crítico
CashSession	 Definido	 No	 No	★ Crítico

 Tablas Adicionales Recomendadas (Opcional)

Tabla Sugerida	Prioridad	Descripción
Customer	 Media	Clientes frecuentes / Facturación
Discount	 Media	Descuentos y promociones
PaymentDetail	 Baja	Detalles de pagos mixtos
SaleReturn	 Baja	Devoluciones de ventas

🎯 Estado Actual y Necesidades para el POS

✅ Lo que Tenemos (Listo para Usar)

1. **✅ Estructura del Proyecto**
 - Next.js 15 con App Router
 - Prisma ORM configurado
 - Supabase PostgreSQL conectado
 - 54 componentes UI de shadcn/ui
2. **✅ Autenticación**
 - NextAuth.js configurado
 - Login funcional
 - Protección de rutas con middleware
 - Roles de usuario (incluyendo CAJA)
3. **✅ Gestión de Inventario**
 - CRUD completo de productos
 - API /api/products funcional
 - Interfaz /admin/inventory implementada
 - Búsqueda y filtros
4. **✅ Schema de Base de Datos**
 - Todas las tablas POS definidas en Prisma
 - Relaciones correctamente establecidas
 - Índices optimizados
5. **✅ Datos de Ejemplo**
 - Script de seed funcional
 - 100+ productos chilenos
 - Tenant y usuario demo

❌ Lo que Falta Implementar (Desarrollo POS)

● Prioridad Alta - Funcionalidad Core POS

1. **API de Ventas (/api/sales)**
 - POST /api/sales - Crear nueva venta
 - GET /api/sales - Listar ventas
 - GET /api/sales/[id] - Detalle de venta
 - PUT /api/sales/[id] - Actualizar venta (cancelar)
 - DELETE /api/sales/[id] - Eliminar venta (soft delete)
2. **API de Sesiones de Caja (/api/cash-session)**
 - POST /api/cash-session/open - Abrir caja
 - POST /api/cash-session/close - Cerrar caja
 - GET /api/cash-session/current - Sesión actual
 - GET /api/cash-session/history - Historial
3. **Interfaz de Punto de Venta (/admin/pos)**
 - Búsqueda rápida de productos (por código de barras o nombre)
 - Carrito de compra con suma automática

- Selección de método de pago
- Cálculo de cambio (efectivo)
- Impresión/descarga de ticket
- Confirmación y registro de venta

4. **Gestión de Caja** (/admin/cash-session)

- Apertura de caja con monto inicial
- Cierre de caja con arqueo
- Cálculo de diferencias
- Resumen de ventas del turno

Prioridad Media - Funcionalidad Complementaria

1. **Historial de Ventas** (/admin/sales)

- Lista de todas las ventas
- Filtros por fecha, método de pago, usuario
- Búsqueda por número de venta
- Exportar a Excel/PDF

2. **Detalle de Venta** (/admin/sales/[id])

- Ver productos vendidos
- Información de pago
- Usuario que realizó la venta
- Opción de reimprimir ticket

3. **Reportes Básicos** (/admin/reports)

- Ventas por período
- Productos más vendidos
- Métodos de pago más usados
- Gráficos con Recharts

Prioridad Baja - Mejoras Futuras

1. **Gestión de Clientes**

- CRUD de clientes
- Historial de compras por cliente
- Facturación

2. **Descuentos y Promociones**

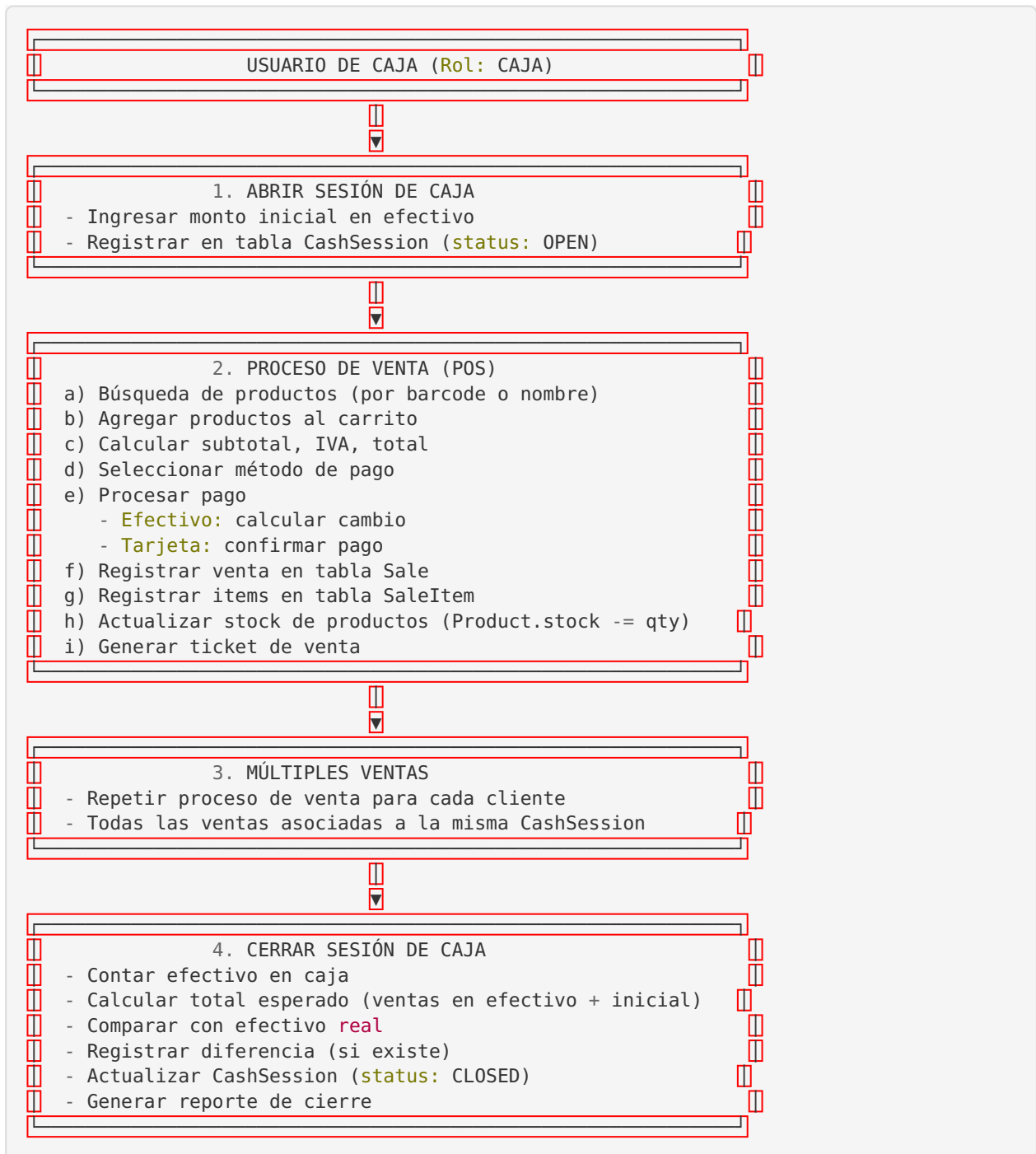
- Aplicar descuentos a productos
- Promociones 2x1, 3x2, etc.
- Códigos de descuento

3. **Ajustes de Inventario Automáticos**

- Reducción automática de stock al vender
 - Alertas de stock bajo en POS
 - Historial de ajustes
-

Arquitectura Recomendada para el Módulo POS

Flujo de Trabajo del POS



Estructura de Archivos Sugerida para POS

app/		
admin/		# NUEVO
pos/		# Interfaz principal del POS
page.tsx		
components/		
product- search .tsx		# Búsqueda de productos
shopping-cart.tsx		# Carrito de compra
payment-modal.tsx		# Modal de pago
receipt.tsx		# Ticket de venta
barcode-scanner.tsx		# Escáner de código de barras
hooks/		
use -pos-cart.ts		# Hook para manejo del carrito
cash- session /		# NUEVO
page.tsx		# Lista de sesiones
open /		
page.tsx		# Abrir caja
close /		
page.tsx		# Cerrar caja
[id]/		
page.tsx		# Detalle de sesión
sales/		# NUEVO
page.tsx		# Lista de ventas
[id]/		
page.tsx		# Detalle de venta
reports/		# NUEVO (opcional)
page.tsx		# Reportes y gráficos
api/		
sales/		# NUEVO
route.ts		# GET (list), POST (create)
[id]/		
route.ts		# GET , PUT, DELETE
cash- session /		# NUEVO
route.ts		# GET (list)
current /		
route.ts		# GET (current session)
open /		
route.ts		# POST (open)
close /		
route.ts		# POST (close)
[id]/		
route.ts		# GET (detail)
products/		
route.ts		# Ya existe
search /		# NUEVO (opcional)
route.ts		# GET con búsqueda avanzada
[id]/		
route.ts		# Ya existe

Componentes a Desarrollar para POS

1. ProductSearch (components/pos/product-search.tsx)

```
// Búsqueda de productos con autocompletado
// - Input con búsqueda en tiempo real
// - Soporte para búsqueda por barcode
// - Lista de resultados con imagen, nombre, precio, stock
// - Selección rápida con teclado (Enter)
```

2. ShoppingCart (components/pos/shopping-cart.tsx)

```
// Carrito de compra
// - Lista de productos agregados
// - Cantidad ajustable (+/-)
// - Eliminar producto
// - Subtotal, IVA, Total
// - Botón "Procesar Pago"
```

3. PaymentModal (components/pos/payment-modal.tsx)

```
// Modal de pago
// - Selector de método de pago (efectivo, débito, crédito, transferencia)
// - Input de monto recibido (solo efectivo)
// - Cálculo automático de cambio
// - Botón "Confirmar Venta"
// - Validaciones
```

4. Receipt (components/pos/receipt.tsx)

```
// Ticket de venta
// - Información del negocio (tenant)
// - Número de venta
// - Fecha y hora
// - Lista de productos con precio y cantidad
// - Subtotal, IVA, Total
// - Método de pago
// - Botón "Imprimir" / "Descargar PDF"
```

5. CashSessionManager (components/cash-session/manager.tsx)

```
// Gestión de sesión de caja
// - Estado actual (abierta/cerrada)
// - Monto inicial
// - Ventas del turno
// - Total esperado
// - Botones "Abrir Caja" / "Cerrar Caja"
```

Hooks Personalizados Recomendados

1. usePOSCart (hooks/use-pos-cart.ts)

```
// Hook para manejo del carrito de compra
interface CartItem {
  product: Product
  quantity: number
  subtotal: number
}

export function usePOSCart() {
  const [items, setItems] = useState<CartItem[]>([])

  const addItem = (product: Product, quantity: number) => { ... }
  const removeItem = (productId: string) => { ... }
  const updateQuantity = (productId: string, quantity: number) => { ... }
  const clearCart = () => { ... }
  const getSubtotal = () => { ... }
  const getTax = () => { ... }
  const getTotal = () => { ... }

  return { items, addItem, removeItem, updateQuantity, clearCart, getSubtotal,
    getTax, getTotal }
}
```

2. useCashSession (hooks/use-cash-session.ts)

```
// Hook para manejo de sesión de caja
export function useCashSession() {
  const [currentSession, setCurrentSession] = useState<CashSession | null>(null)

  const openSession = async (initialAmount: number) => { ... }
  const closeSession = async (finalAmount: number) => { ... }
  const getCurrentSession = async () => { ... }

  return { currentSession, openSession, closeSession, getCurrentSession }
}
```



Ejemplo de Flujo de API para Crear Venta

Endpoint: POST /api/sales

Request Body:

```
{
  "items": [
    {
      "productId": "clxxxxx1",
      "quantity": 2,
      "unitPrice": 1520.00,
      "unitCost": 912.00
    },
    {
      "productId": "clxxxxx2",
      "quantity": 1,
      "unitPrice": 8990.00,
      "unitCost": 5394.00
    }
  ],
  "paymentMethod": "CASH",
  "cashReceived": 15000.00,
  "cashSessionId": "clxxxxx3"
}
```

Lógica del Backend:

```
// 1. Validar sesión de caja activa
// 2. Validar productos y stock disponible
// 3. Calcular subtotal, tax, total
// 4. Generar número de venta único
// 5. Crear registro en tabla Sale
// 6. Crear registros en tabla SaleItem
// 7. Actualizar stock de productos (Product.stock -= quantity)
// 8. Retornar venta creada con items
```

Response:

```
{
  "id": "clxxxxx4",
  "saleNumber": "VT-0001",
  "subtotal": 12030.00,
  "tax": 2285.70,
  "total": 14315.70,
  "paymentMethod": "CASH",
  "cashReceived": 15000.00,
  "change": 684.30,
  "status": "COMPLETED",
  "items": [
    {
      "id": "clxxxxx5",
      "productId": "clxxxxx1",
      "productName": "Coca-Cola Original 1.5L",
      "quantity": 2,
      "unitPrice": 1520.00,
      "subtotal": 3040.00
    },
    {
      "id": "clxxxxx6",
      "productId": "clxxxxx2",
      "productName": "Miel Natural 500g",
      "quantity": 1,
      "unitPrice": 8990.00,
      "subtotal": 8990.00
    }
  ],
  "createdAt": "2025-10-25T14:30:00.000Z"
}
```

Consideraciones de Seguridad

Validaciones Importantes

1. Sesión de Caja Activa

- Solo se pueden crear ventas si hay una sesión de caja abierta
- Validar que la sesión pertenece al tenant del usuario

2. Stock Disponible

- Validar que hay suficiente stock antes de crear la venta
- Implementar manejo de concurrencia (transacciones)

3. Permisos de Usuario

- Solo usuarios con rol `CAJA` o `ADMIN` pueden acceder al POS
- Implementar middleware de autorización por rol

4. Integridad de Datos

- Usar transacciones de Prisma para operaciones críticas
- Validar cálculos de totales en el backend
- No confiar en cálculos del frontend

5. Multi-tenencia

- Todas las queries deben filtrar por `tenantId`
- Usar middleware para inyectar `tenantId` automáticamente

Pasos Siguientes para Desarrollo

Fase 1: APIs Core (1-2 días)

- [] Implementar `POST /api/sales` (crear venta)
- [] Implementar `GET /api/sales` (listar ventas)
- [] Implementar `GET /api/sales/[id]` (detalle de venta)
- [] Implementar `POST /api/cash-session/open` (abrir caja)
- [] Implementar `POST /api/cash-session/close` (cerrar caja)
- [] Implementar `GET /api/cash-session/current` (sesión actual)
- [] Añadir validaciones y manejo de errores
- [] Implementar transacciones de Prisma

Fase 2: Interfaz POS (2-3 días)

- [] Crear página `/admin/pos`
- [] Implementar componente `ProductSearch`
- [] Implementar componente `ShoppingCart`
- [] Implementar componente `PaymentModal`
- [] Implementar hook `usePOSCart`
- [] Conectar con APIs de backend
- [] Añadir validaciones de frontend
- [] Implementar notificaciones con `toast`

Fase 3: Gestión de Caja (1 día)

- [] Crear página `/admin/cash-session`
- [] Crear página `/admin/cash-session/open`
- [] Crear página `/admin/cash-session/close`
- [] Implementar componente `CashSessionManager`
- [] Implementar hook `useCashSession`
- [] Añadir protección de rutas (solo CAJA/ADMIN)

Fase 4: Historial y Reportes (1-2 días)

- [] Crear página `/admin/sales`
- [] Crear página `/admin/sales/[id]`
- [] Implementar filtros y búsqueda
- [] Implementar paginación
- [] Crear página `/admin/reports` (opcional)
- [] Añadir gráficos con Recharts

Fase 5: Testing y Refinamiento (1 día)

- [] Ejecutar seed de datos
- [] Probar flujo completo de venta
- [] Probar apertura/cierre de caja
- [] Validar cálculos de totales
- [] Verificar actualización de stock
- [] Probar con múltiples usuarios

- [] Optimizar rendimiento

Recursos y Documentación

Documentación del Proyecto

- **Ubicación:** `/home/ubuntu/github_repos/crtlpyme-mvp-temp/`
- **Archivos relevantes:**
 - `Analisis_Completo_Proyecto_CRTLPyme.md` - Análisis previo del proyecto
 - `FASE-1-PLAN.md` - Plan de Fase 1 (completada parcialmente)
 - `FASE-2-PLAN.md` - Plan de Fase 2 (POS Core) - ★ **Relevante**
 - `ROADMAP.md` - Roadmap completo del proyecto
 - `INSTRUCCIONES_MVP.md` - Instrucciones generales del MVP

Tecnologías y Referencias

Tecnología	Documentación
Next.js 15	https://nextjs.org/docs
React 19	https://react.dev/
Prisma 6	https://www.prisma.io/docs
NextAuth.js	https://next-auth.js.org/
shadcn/ui	https://ui.shadcn.com/
Radix UI	https://www.radix-ui.com/
Tailwind CSS	https://tailwindcss.com/
Zod	https://zod.dev/
React Hook Form	https://react-hook-form.com/

Checklist de Preparación Completada

- [x] Localización del proyecto
- [x] Revisión de estructura de carpetas
- [x] Análisis de rutas implementadas
- [x] Inventario de componentes UI disponibles
- [x] Revisión de schema de base de datos
- [x] Identificación de tablas existentes y faltantes
- [x] Análisis de datos de ejemplo disponibles
- [x] Definición de arquitectura recomendada

- [x] Identificación de APIs necesarias
- [x] Creación de checklist de desarrollo

Resumen Ejecutivo

El Proyecto Está Listo Para:

- Desarrollo del sistema POS
- Todas las tablas necesarias están definidas en el schema de Prisma
- Componentes UI disponibles y listos para usar
- Autenticación y gestión de usuarios implementada
- Inventario de productos funcional con CRUD completo

Se Necesita Implementar:

- APIs de ventas (`/api/sales`)
- APIs de sesiones de caja (`/api/cash-session`)
- Interfaz de punto de venta (`/admin/pos`)
- Gestión de caja (`/admin/cash-session`)
- Historial de ventas (`/admin/sales`)

Tiempo Estimado de Desarrollo:

- **Core POS (APIs + Interfaz):** 3-4 días
- **Gestión de Caja:** 1 día
- **Historial y Reportes:** 1-2 días
- **Testing y Refinamiento:** 1 día
- **Total:** 6-8 días laborables

Próximo Paso Inmediato:

Comenzar con la implementación de las APIs de ventas en `/api/sales/route.ts`, siguiendo el ejemplo de flujo documentado en este reporte.

Documento generado el: 25 de Octubre, 2025

Por: DeepAgent - Análisis Completo de Preparación POS CRTLPyme

Versión: 1.0.0